Event Management Platform

Technical & Functional Specification

Prepared by: UXITech

Client: Siva Rudra Foundations

Purpose: Complete reference for the development team —
problem statement, solution, and every feature with implementation examples.

Version 1.0 — Internal Development Document

1. Problem Statement

Siva Rudra Foundations organizes live pageant-style events (ramp walks, dance competitions, talent shows) across categories such as Kids, Teen, Miss, Ms and Mr. Today this runs manually:

- Registrations are collected offline/ad-hoc — no central record, no automatic confirmation.
- Judges score on paper — slow, error-prone, and scores can't combine across multiple judges or across a two-day event automatically.
- There is no live, trustworthy way to show a running leaderboard on the stage LED screen.
- There is no public record of past events, winners or scores — new participants have no easy way to trust the brand before registering.
- Judges scoring later rounds can be influenced by seeing earlier scores (bias risk).

We are replacing this manual process with one connected platform that handles registration, payment tracking, day-wise scoring by multiple roles, automatic score aggregation, and a real-time public leaderboard — while keeping judges “blind” to information that could bias their scoring.

2. Solution Overview

A single platform with four connected surfaces, all reading from and writing to one shared backend:

Surface	Used by	Core job
Public Website	General public	Discover brand, view past event results, browse & register for upcoming events
Admin Panel	Siva Rudra Foundations	Create events/categories/rounds, manage registrations & payments, score Traditional/Discipline, manage judges, view all data
Judges Panel	Judges (assigned per event)	Score only their assigned round, for contestants in their assigned event, by ID only
Contestant Dashboard	Registered participants	View own profile, payment status, live rank

A read-only **Live Leaderboard** view, rendered from the same backend, is displayed on the stage LED screen.

3. User Roles & Permissions

Role	Can do	Cannot do
Public visitor	View site, register for events	Access any dashboard
Admin (client)	Full control of events, registrations, payments, Traditional/Discipline scoring, judge accounts, edit authorization, all data	— (top authority)
Judge	Log in to assigned event only; score assigned round for contestants shown by ID	See other rounds/days' scores; self-edit after submit; see contestant names
Contestant	Log in via mobile+OTP; view own profile & live rank	View other contestants' detailed scores; edit own data after submission

4. Feature Specifications

Each feature includes what it does, why, and a worked example so the logic is unambiguous during development.

4.1 Public Website

- Home/About page — brand story, mission, photos
- **Past Events** — grid of completed events; opening one shows name, location, date, winners with final scores, and 4–5 photos
- **Upcoming Events** — grid with event logo + short description; clicking opens the registration form

Example: Public user opens “Nellore Nirajan” under Past Events → sees Location: Nellore | Date: 12 Jul 2026 | Winners — Ms: SRF-NLR-MS-0007 (92.5), Mr: SRF-NLR-MR-0003 (89.0), Kids: SRF-NLR-K-0011 (95.0) — plus a 5-photo gallery.

4.2 Dynamic Registration System

- Fixed base fields: Name, Mobile, Location, Gender, Email, Age, DOB
- Admin adds custom fields per event (e.g. Instagram handle, height)
- Registrant selects one category, defined per event by admin — not hardcoded
- On submit: record appears in Admin Panel + WhatsApp notification sent to client

Implementation note: *store custom fields as flexible key-value pairs per event, not fixed DB columns, so admin can add new fields without a code change.*

Example — event creation defines the form:

```
{
  "event": "Nellore Nirajan",
  "categories": ["Kids", "Teen", "Miss", "Ms", "Mr"],
  "base_fields": ["name", "mobile", "location", "gender", "email", "age", "dob"],
  "custom_fields": [
    { "label": "Instagram Handle", "type": "text", "required": false },
    { "label": "Height (cm)", "type": "number", "required": true }
  ]
}
```

4.3 Payment Status & Unique Contestant ID

- Admin panel shows a Paid / Not Paid toggle per registrant (covers online and offline payment)
- Marking “Paid” auto-generates a unique contestant ID

ID format: SRF-[EVENT CODE]-[CATEGORY CODE]-[SEQUENCE]

- SRF — fixed brand prefix
- [EVENT CODE] — short code set by admin per event (e.g. NLR for Nellore)
- [CATEGORY CODE] — K / T / MS / MRS / MR
- [SEQUENCE] — zero-padded, increments per category per event (0001, 0002 …)

Example: 3rd Kids registrant marked “Paid” in Nellore Nirajan → ID generated: **SRF-NLR-K-0003**. This ID is what judges and the leaderboard show — never the contestant's name.

4.4 Admin Panel

- **Event Builder:** create event; define categories; define rounds per category; set event code, dates, location, logo/description
- **Registrations view:** table of all registrants, filterable by category/payment status
- **Payment toggle:** Paid/Not Paid, triggers ID generation
- **Scoring — Traditional & Discipline:** admin enters these directly
- **Judge management:** create judge accounts (email+password), assign to event + category + round

- **Edit authorization:** judges can't self-edit submitted scores; admin can unlock a specific entry
- **Full visibility:** admin sees every score, every round, every day, at any time

Example — rounds configuration for two categories:

```
{ "category": "Kids", "rounds": [
  { "name": "Traditional", "max_marks": 10, "scored_by": "admin", "day": 1 },
  { "name": "Discipline",  "max_marks": 10, "scored_by": "admin", "day": 1 },
  { "name": "Talent", "max_marks": 20, "scored_by": "judge", "day": 2,
    "sub_criteria": [
      { "name": "Face Expression", "max_marks": 10 },
      { "name": "Dressing", "max_marks": 10 } ] } ] }
```

```
{ "category": "Ms", "rounds": [
  { "name": "Traditional", "max_marks": 10, "scored_by": "admin", "day": 1 },
  { "name": "Discipline",  "max_marks": 10, "scored_by": "admin", "day": 1 },
  { "name": "Talent",  "max_marks": 20, "scored_by": "judge", "day": 2 },
  { "name": "Western", "max_marks": 20, "scored_by": "judge", "day": 2 } ] }
```

This structure (round name, max marks, who scores it, which day, optional sub-criteria) must be fully configurable per event — never hardcoded.

4.5 Judges Panel

- Judge logs in with admin-issued email/password
- Dashboard opens automatically to their assigned event
- Sees contestants by unique ID only, in their assigned category/round only
- Enters marks per sub-criterion (if defined)
- Cannot see other rounds', other days', or other judges' scores, or contestant names
- Once submitted, score is locked until admin grants edit access

Example — Day 2, Talent round, judge's view:

Contestant ID	Face Expression (/10)	Dressing (/10)
SRF-NLR-K-0001	__	__
SRF-NLR-K-0002	__	__
SRF-NLR-K-0003	__	__

No Traditional/Discipline column appears here — those were Day 1, admin-entered, and intentionally hidden from this judge.

4.6 Scoring Engine & Automatic Aggregation

- Every round's score is stored separately: event, category, contestant ID, round, day, judge (if applicable)
- Total score = sum of all rounds, calculated automatically
- Day 1 scores visible to admin any time, hidden from Day-2 judges
- Combined total appears on the leaderboard once all applicable rounds are scored

Example — full score buildup for one contestant:

Round	Day	Scored by	Max	Score
Traditional	1	Admin	10	8
Discipline	1	Admin	10	8
Talent	2	Judges (avg)	20	17
Western	2	Judges (avg)	20	16
Total	—	—	60	49

This total (49/60) is what appears on the live leaderboard — the judges who scored Talent/Western never saw the Day-1 8+8 while scoring.

4.7 Multiple Judges & Averaging

- Admin sets how many judges (2–5) are assigned per round, per event
- Once all assigned judges submit for a contestant's round, system computes the plain average
- Optional toggle: “Drop highest and lowest” before averaging — off by default
- System waits for all assigned judges before finalizing, so the leaderboard doesn't jump around mid-scoring

Example: Talent round, 3 judges. Scores: 18, 16, 17. Plain average = **17.0**. With “drop highest/lowest” ON: drop 18 & 16 → final = **17** (with 5 judges this becomes a genuine trimmed average).

4.8 Live Leaderboard (Stage LED Display)

- Read-only, auto-updating screen — rank-ordered by total score, per category

- Separate leaderboard per category (Kids, Teen, Miss, Ms, Mr)
- Shows contestant ID (not name) and total score
- Rendered as full-screen browser view on a laptop/mini-PC connected to the venue LED screen

Example — Kids leaderboard mid-event:

Rank	ID	Total Score
1	SRF-NLR-K-0011	49
2	SRF-NLR-K-0003	46
3	SRF-NLR-K-0007	44

4.9 Contestant Dashboard

- Login via mobile number + OTP — no password to manage
- View profile, registration/payment status, current live rank

Example: Contestant SRF-NLR-K-0003 logs in with registered mobile + OTP and sees: “Your current rank: 2nd place — Kids category — Total Score: 46/60.”

4.10 WhatsApp Notifications

- Every new registration triggers an instant WhatsApp message to the client
- Uses the official WhatsApp Business API (Twilio / Gupshup / Meta Cloud API) — not an unofficial automation

Example message payload (conceptual):

```
New Registration – Nellore Nirajan
Name: [Contestant Name]
Category: Kids
Mobile: [Mobile Number]
Status: Payment Pending
```

4.11 Security

- Judges: email + admin-generated password, forced reset on first login, session scoped server-side to assigned event/category/round
- Contestants: mobile + OTP login (passwordless)
- Admin: password + optional 2FA — this account can see and edit everything
- All traffic over HTTPS
- Every score submission/edit logged (who, what, when) for auditability

5. Core Data Model (Entities)

Entity	Key fields
Event	id, name, code, location, date(s), logo, description, status
Category	id, event_id, name, code
Round	id, category_id, name, max_marks, scored_by, day, sub_criteria[]
Registration	id, event_id, category_id, base_fields{}, custom_fields{}, payment_status, contestant_id
Contestant	id (unique ID e.g. SRF-NLR-K-0003), registration_id, mobile
Score	id, contestant_id, round_id, judge_id, sub_scores{}, value, submitted_at, locked
JudgeAccount	id, name, email, password_hash, assigned_event_id, assigned_category_id, assigned_round_id
AdminUser	id, name, email, password_hash

6. End-to-End Example Walkthrough — “Nellore Nirajan”

1. Admin creates event Nellore Nirajan (code NLR), adds 5 categories, defines rounds per category (Kids: Traditional, Discipline, Talent; Teen/Miss/Ms/Mr: + Western), sets Day 1 = Traditional + Discipline, Day 2 = Talent + Western.

2. Event appears on the public website under Upcoming Events.

3. A parent registers their child under Kids category — fills base + custom fields — submits.

4. Admin panel shows the new registration instantly; client's WhatsApp receives a notification.

5. Client collects payment offline at the venue → admin toggles Paid → system generates ID SRF-NLR-K-0003.

6. Day 1: Admin enters Traditional (8/10) and Discipline (8/10) scores directly for each contestant.

7. Day 2: Admin creates judge logins, assigns 3 judges to Kids → Talent round. Judges see only “Nellore Nirajan,” contestants by ID only, and enter Face Expression + Dressing marks — without seeing Day 1 scores.

8. Once all 3 judges submit for a contestant, the system averages their Talent scores.

9. System auto-computes Total = Traditional + Discipline + averaged Talent.

10. Live leaderboard on the stage LED updates automatically, ranked within the Kids category board.

11. Contestant's parent logs into the Contestant Dashboard via mobile + OTP and sees the child's live rank.

12. After the event, admin marks it Past → public website shows Nellore Nirajan under Past Events with winners, final scores, and photos.

7. Suggested Tech Stack

Layer	Recommendation	Why
Frontend (all 4 surfaces)	React (or Next.js)	Component reuse across admin/judge/contestant/public surfaces
Backend	Node.js (Express/NestJS)	REST/WebSocket APIs, real-time score push

Layer	Recommendation	Why
Database	PostgreSQL (or MongoDB if schema stays flexible)	Relational integrity for events/categories/rounds/scores
Real-time updates	WebSockets (Socket.io) or short-interval polling	Live leaderboard refresh without manual reload
WhatsApp	Official WhatsApp Business API (Twilio/Gupshup/Meta)	Reliable, won't get banned
Auth	JWT sessions; OTP via SMS/WhatsApp for contestants	Matches the passwordless requirement
Hosting	VPS/Cloud (not basic shared hosting)	Needed for Node.js backend + real-time features

8. Non-Functional Requirements

- **Live-event reliability:** scoring + leaderboard must stay responsive with concurrent judges submitting during a live show — load-test before the first real event
- **Mobile-first:** registration form and contestant dashboard will mostly be used on phones
- **Data integrity:** a locked score must not be editable without a logged, admin-authorized action
- **Bias prevention:** “judges can't see prior rounds/days” is a hard requirement, enforced at the API level — not just a UI nicety

9. Open Items to Confirm Before/During Build

- Exact categories and rounds for the first real event (to seed initial configuration)
- Whether ID sequence numbering resets per event or runs as one continuous counter across all events
- Final choice of WhatsApp API provider (cost/volume dependent)
- Whether leaderboard ties need an explicit tie-break rule

End of specification — UXITech, Vijayawada — uxitech.in